Bitácora Lab S11 · Mexi Banco: una verdad, varias copias

1. El punto de partida

1.1 Banco levantado

Comando: `docker compose up -d` y `docker compose ps` (trece contenedores *healthy*).

Captura 1.1 — docker compose ps con todos los contenedores healthy

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose ps
NAME                          IMAGE                          COMMAND                  SERVICE         CREATED              STATUS
                  PORTS
mexi-banco-v06-cuenta-1       mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          About a minute ago   Up About
a minute (healthy)   8080/tcp
mexi-banco-v06-cuenta-2       mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          About a minute ago   Up About
a minute (healthy)   8080/tcp
mexi-banco-v06-cuenta-3       mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          About a minute ago   Up About
a minute (healthy)   8080/tcp
mexi-banco-v06-db-1           postgres:17-alpine             "docker-entrypoint.s…"   db              26 minutes ago       Up 26 min
utes (healthy)       5432/tcp
mexi-banco-v06-discovery-1    mexi-banco-v06-discovery       "java -jar app.jar"      discovery       About a minute ago   Up About
a minute (healthy)   0.0.0.0:8761->8761/tcp, [::]:8761->8761/tcp
mexi-banco-v06-gateway-1      mexi-banco-v06-gateway         "java -jar app.jar"      gateway         About a minute ago   Up About
a minute (healthy)   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-v06-mongo-1        mongo:8.0                      "docker-entrypoint.s…"   mongo           26 minutes ago       Up 26 min
utes (healthy)       127.0.0.1:27017->27017/tcp
mexi-banco-v06-movimiento-1   mexi-banco-v06-movimiento      "java -jar app.jar"      movimiento      About a minute ago   Up About
a minute (healthy)   8080/tcp
mexi-banco-v06-notificacion-1 mexi-banco-v06-notificacion    "java -jar app.jar"      notificacion    About a minute ago   Up About
a minute (healthy)   8080/tcp
mexi-banco-v06-rabbitmq-1     rabbitmq:4-management          "docker-entrypoint.s…"   rabbitmq        26 minutes ago       Up 26 min
utes (healthy)       0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
mexi-banco-v06-redis-1        redis:8-alpine                 "docker-entrypoint.s…"   redis           26 minutes ago       Up 26 min
utes (healthy)       127.0.0.1:6379->6379/tcp
mexi-banco-v06-spei-1         mexi-banco-v06-spei            "java -jar app.jar"      spei            About a minute ago   Up About
a minute (healthy)   8080/tcp
mexi-banco-v06-transferencia-1 mexi-banco-v06-transferencia  "java -jar app.jar"      transferencia   About a minute ago   Up About
a minute (healthy)   8080/tcp
```

1.2 Redis y MongoDB contestan

Comandos: redis-cli ping (esperado PONG) y db.avisos.countDocuments() (esperado 0).

Captura 1.2 — PONG y 0

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli ping
PONG

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.countDocuments()'
0
```

1.3 Cuentas y primera transferencia

Ana y Beto dieron error al crearlas puesto que ya existían, Carla (...003, 300) y una transferencia de 200 de Ana a Beto.

Captura 1.3 — altas (201) y transferencia COMPLETADA

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' \
  -d '{"clabe":"002180000000000001","titular":"Ana","saldoInicial":1000}'
HTTP/1.1 409 Conflict
transfer-encoding: chunked
X-Instancia: 2b01c1112a50
Content-Type: application/problem+json
Date: Fri, 06 Nov 2026 16:59:21 GMT

{"type":"about:blank","title":"Conflict","status":409,"detail":"Ya existe una cuenta con la CLABE 002180000000000001","instance":"/cuentas"}
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' \
  -d '{"clabe":"002180000000000002","titular":"Beto","saldoInicial":500}'
HTTP/1.1 409 Conflict
transfer-encoding: chunked
X-Instancia: 0cf30959a6a6
Content-Type: application/problem+json
Date: Fri, 06 Nov 2026 16:59:52 GMT

{"type":"about:blank","title":"Conflict","status":409,"detail":"Ya existe una cuenta con la CLABE 002180000000000002","instance":"/cuentas"}
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' \
  -d '{"clabe":"002180000000000003","titular":"Carla","saldoInicial":300}'
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: c6fb2dfedd1f
Content-Type: application/json
Date: Fri, 06 Nov 2026 17:00:02 GMT

{"id":8,"clabe":"002180000000000003","titular":"Carla","saldo":300}
```

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose exec db psql -U mexibanco -d cuenta -c "SELECT * FROM cuenta;"
 id |       clabe        | saldo  | titular | version
----+--------------------+--------+---------+---------
  1 | S071791566166      |   0.00 | S07     |       0
  2 | S071791585825      |   0.00 | S07     |       0
  3 | S071791609089      |   0.00 | S07     |       0
  4 | 002180009161154301 | 850.00 | Ana     |      16
  5 | 002180009161154302 | 750.00 | Beto    |       2
  6 | 002180000000000001 | 854.00 | Ana     |      10
  7 | 002180000000000002 | 646.00 | Beto    |       6
  8 | 002180000000000003 | 300.00 | Carla   |       0
(8 rows)
```

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ curl -s -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":200}'
{"id":15,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":200,"fecha":"2026-11-06T17:05:41.405074633Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":654.00,"intentosDeCompensacion":0,"actualizada":"2026-11-06T17:05:42.867202301Z","eventoId":"c6df7b5e-bb2f-41d9-8dfe-ff89e2100e67"}
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose exec db psql -U mexibanco -d cuenta -c "SELECT * FROM cuenta;"
 id |       clabe        | saldo  | titular | version
----+--------------------+--------+---------+---------
  1 | S071791566166      |   0.00 | S07     |       0
  2 | S071791585825      |   0.00 | S07     |       0
  3 | S071791609089      |   0.00 | S07     |       0
  4 | 002180009161154301 | 850.00 | Ana     |      16
  5 | 002180009161154302 | 750.00 | Beto    |       2
  8 | 002180000000000003 | 300.00 | Carla   |       0
  6 | 002180000000000001 | 654.00 | Ana     |      11
  7 | 002180000000000002 | 846.00 | Beto    |       7
(8 rows)
```

1.4 Medición "antes": cada consulta va a Postgres

Comandos: ciclo de 10 curl al saldo de Ana, docker compose logs cuenta | grep -c "leida de Postgres" y redis-cli INFO stats | grep keyspace.

Captura 1.4a — las 10 respuestas

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ for i in $(seq 1 10); do
  curl -s http://localhost:8080/api/cuentas/002180000000000001; echo
done
{"clabe":"002180000000000001","titular":"Ana","saldo":654.00,"leidaEn":"2026-11-06T17:07:23.785635851Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":654.00,"leidaEn":"2026-11-06T17:07:23.998298557Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":654.00,"leidaEn":"2026-11-06T17:07:24.498365679Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":654.00,"leidaEn":"2026-11-06T17:07:24.727626069Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":654.00,"leidaEn":"2026-11-06T17:07:24.930837453Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":654.00,"leidaEn":"2026-11-06T17:07:25.076638037Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":654.00,"leidaEn":"2026-11-06T17:07:25.289062789Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":654.00,"leidaEn":"2026-11-06T17:07:25.452961995Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":654.00,"leidaEn":"2026-11-06T17:07:25.688279369Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":654.00,"leidaEn":"2026-11-06T17:07:25.918252013Z"}
```

Captura 1.4b — conteo de "leida de Postgres" y keyspace_hits / keyspace_misses

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose logs cuenta | grep -c "leida de Postgres"
12

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli INFO stats | grep keyspace
keyspace_hits:0
keyspace_misses:0
```

Consultas hechas	Lecturas a Postgres	keyspace_hits	keyspace_misses
10	12	0	0

Conclusión del "antes": esta captura nos dice que la aplicación, hasta este punto, está yendo directamente a la base de datos en cada consulta. El conteo de logs demuestra que las 1o consultas que se hicieron llegan hasta la base de datos principal a trabajar en cada lectura, mientras que las estadísticas de Redis en ceros demuestra que precisamente la caché de redis está inactiva y no está ayudando a aligerar la carga de las consultas, por ahora.

2. La caché del saldo en Redis

2.1 Cambios en código

ConfiguracionCache.java — `@EnableCaching` y el `RedisCacheManagerBuilderCustomizer`.

@EnableCaching: este funciona como el interruptor principal, que enciende el sistema de caché en Spring Boot. Sin esto, Spring solo ignora cualquier otra anotación, como @Cacheable o @CacheEvict, y se seguiría consultando PostgreSQL

entryTtl(ttl): Se agrega para definir el tiempo de caducidad de los datos, sin esta línea los saldos se quedarían guardados para siempre, provocando que los clientes vean saldos viejos e incorrectos si el dinero cambió ya.

Jackson2JsonRedisSerializer: esta sección le dice a Spring que guarde los datos en la caché utilizando texto claro en formato Json, de forma que podamos entenderlo al consultarlo, ya que Spring guarda los objetos Java en código binario por defecto.

transactionAware(): esto es muy importante ya que obliga a Redis a esperar la confirmación (commit) de PostgreSQL antes de borrar o actualizar la caché tras un movimiento. Sin esto, la caché podría borrarse una fracción de segundo antes de que la base de datos termine de guardar el nuevo saldo, permitiendo que una consulta rápida se cuele y vuelva a guardar un saldo desactualizado (dinero fantasma) en Redis.

ConsultaDeSaldo.java — @Cacheable(cacheNames = CUENTAS, key = "#clabe") Aquí le decimos a Spring que guarde el dato usando la clabe. En Redis, la llave se guarda juntando el nombre de la caché y el número, así que queda con un formato tipo cuentas::002180000000000001.

- **En un Hit (sí está en caché):** Spring encuentra la llave en Redis, corta la ejecución del método ahí mismo y regresa el saldo de inmediato. Con esto nos saltamos la consulta a Postgres y le quitamos carga a la base de datos.
- **En un Miss (no está en caché):** Como no encuentra la llave en Redis, el método sigue su flujo normal. Va hasta Postgres por el saldo real, lo devuelve, y antes de terminar, guarda una copia en Redis para que la próxima vez sí sea un hit.

CuentaService.java — @CacheEvict en cargar y abonar. Le pusimos el @CacheEvict a los métodos que mueven dinero para que, en cuanto se haga un cargo o un abono, se borre en automático el saldo viejo que estaba guardado en Redis. De esta forma, la siguiente vez que alguien consulte su saldo, a fuerza será un "miss", obligando al sistema a traer el saldo fresco directo de la base de datos.

¿Por qué CuentaService.consultar NO se cachea? Básicamente por cómo funciona Spring con sus "proxies" (que son como intermediarios). Si le pongo el @Cacheable a un método y luego otro método de esa misma clase lo manda a llamar (por ejemplo, si abonar llama a this.consultar() para checar si hay fondos), la llamada se hace directa y se salta al intermediario. Al saltárselo, Spring ignora la anotación y la caché simplemente no funciona. Por eso tuvimos que sacar la consulta a un caso de uso aparte (ConsultaDeSaldo), para que la llamada venga "desde afuera", pase por el proxy de Spring y la caché sí haga lo que debe.

2.2 Las tres copias comparten la caché

Se reconstruyó con `docker compose up --build -d cuenta` y se pidió 6 veces el saldo de Ana.

Mi predicción antes de correrlo (¿cuántas lecturas a Postgres?): Se hará una sola lectura inicial directamente a Postres y posteriormente ya se leerá de la memoria caché.

Captura 2.2a — 6 respuestas con X-Instancia distinta y el mismo leidaEn

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ for i in 1 2 3 4 5 6; do
  curl -s -D - http://localhost:8080/api/cuentas/002180000000000001 | grep -iE 'x-instancia|leidaEn'
done
X-Instancia: a049a895660b
{"clabe":"002180000000000001","titular":"Ana","saldo":654.00,"leidaEn":"2026-11-06T20:59:43.194618134Z"}
X-Instancia: d761e21f3b02
{"clabe":"002180000000000001","titular":"Ana","saldo":654.00,"leidaEn":"2026-11-06T20:59:43.194618134Z"}
X-Instancia: b95a9f42be2e
{"clabe":"002180000000000001","titular":"Ana","saldo":654.00,"leidaEn":"2026-11-06T20:59:43.194618134Z"}
X-Instancia: a049a895660b
{"clabe":"002180000000000001","titular":"Ana","saldo":654.00,"leidaEn":"2026-11-06T20:59:43.194618134Z"}
X-Instancia: a049a895660b
{"clabe":"002180000000000001","titular":"Ana","saldo":654.00,"leidaEn":"2026-11-06T20:59:43.194618134Z"}
X-Instancia: b95a9f42be2e
{"clabe":"002180000000000001","titular":"Ana","saldo":654.00,"leidaEn":"2026-11-06T20:59:43.194618134Z"}
```

Captura 2.2b — conteo de lecturas a Postgres en los últimos 30 s

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose logs --since 30s cuenta | grep -c "leida de Postgres"
1
```

Captura 2.2c — redis-cli GET cuentas::002180000000000001 (JSON) y TTL

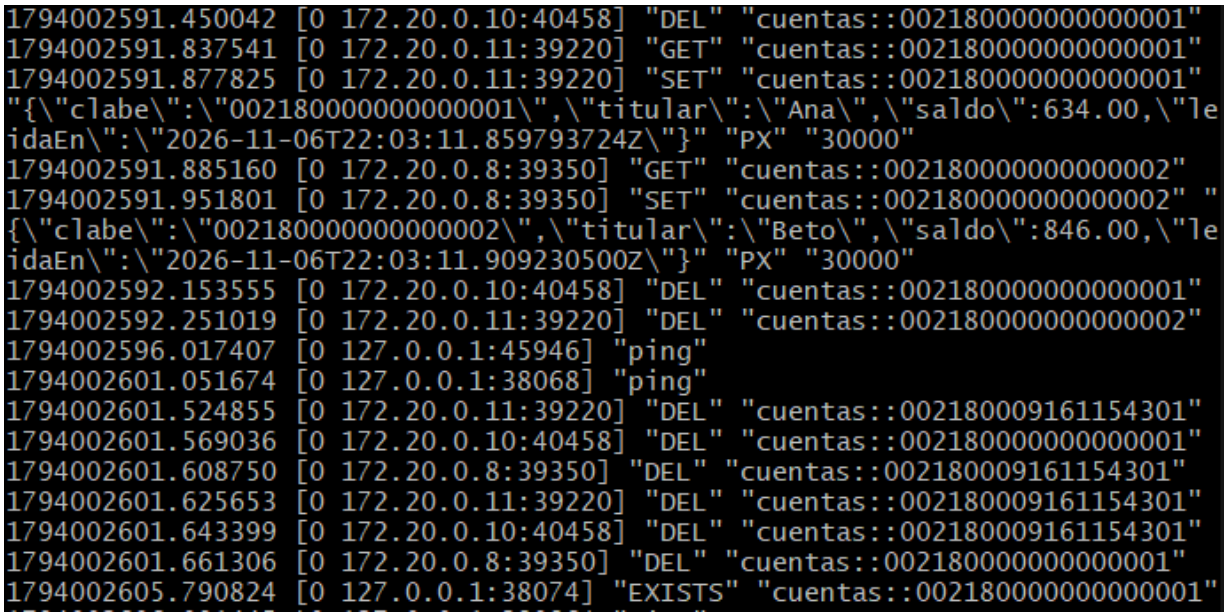
```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli GET cuentas::002180000000000001
```

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ curl -s http://localhost:8080/api/cuentas/002180000000000001 > /dev/null && docker comp
ose exec -T redis redis-cli TTL cuentas::002180000000000001
29
```

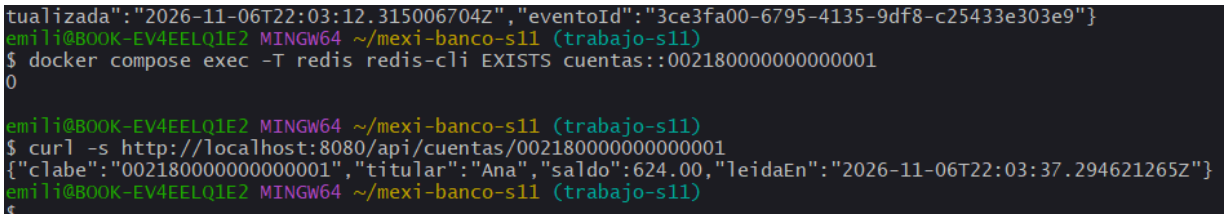
2.3 La copia se borra al transferir

Terminal 2: redis-cli MONITOR. Terminal 1: transferir 10 de Ana a Beto, EXISTS cuentas::002180000000000001 y volver a consultar.

Captura 2.3a — MONITOR mostrando GET, DEL (Ana), DEL (Beto), GET (miss) y SET ... PX 30000



Captura 2.3b — EXISTS = 0 y el saldo nuevo con leidaEn nuevo



En el MONITOR se ve primero un GET de la saga que revisa la cuenta, luego un DEL de la clave de Ana y otro de la de Beto, cada uno después del commit del cargo y del abono respectivamente. Después, mi consulta hace un GET (miss), el método lee Postgres y Spring hace SET ... PX 30000 con el saldo nuevo. EXISTS dio 0 justo después de la transferencia, lo que confirma que la copia se borra en lugar de actualizarse.

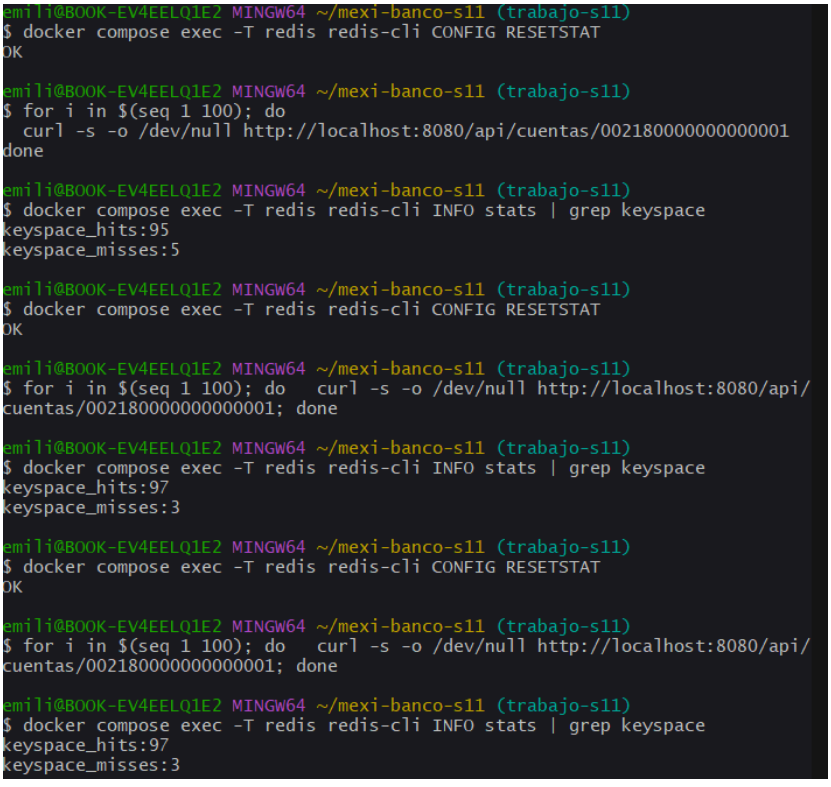
3. Tus medidas

3.1 Hit ratio (hits / (hits + misses))

- **Escenario A:** 100 consultas seguidas del saldo de Ana.
- **Escenario B:** lo mismo, pero con una transferencia de 1 peso de Ana a Beto cada 10 consultas.

Escenario	Corrida	hits	misses	Hit ratio
A	1	95	5	95%
A	2	97	2	97%
A	3	97	3	97%
B	1	98	22	81.7%
B	2	99	21	82.5%
B	3	97	23	80.8%

Captura 3.1 — keyspace_hits / keyspace_misses de A y B



Captura 3.1 — keyspace_hits / keyspace_misses de A y B

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli CONFIG RESETSTAT
OK

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ for i in $(seq 1 100); do
  # 1. Consulta el saldo de Ana
  curl -s -o /dev/null http://localhost:8080/api/cuentas/0021800000000000001

  # 2. Si la consulta es múltiplo de 10, dispara la transferencia
  if [ $((i % 10)) -eq 0 ]; then
    curl -s -o /dev/null -X POST http://localhost:8080/api/transferencias \
      -H 'Content-Type: application/json' \
      -d '{"claveOrigen":"0021800000000000001","claveDestino":"002180000000000
002","monto":1}'
  fi
done

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli INFO stats | grep keyspace
keyspace_hits:98
keyspace_misses:22

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli CONFIG RESETSTAT
OK

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ for i in $(seq 1 100); do
  curl -s -o /dev/null http://localhost:8080/api/cuentas/0021800000000000001;

  if [ $((i % 10)) -eq 0 ]; then     curl -s -o /dev/null -X POST http://loca
lhost:8080/api/transferencias       -H 'Content-Type: application/json'
 -d '{"claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002",
"monto":1}';   fi; done

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli INFO stats | grep keyspace
keyspace_hits:99
keyspace_misses:21

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli CONFIG RESETSTAT
OK

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ for i in $(seq 1 100); do   curl -s -o /dev/null http://localhost:8080/api/
cuentas/0021800000000000001;      if [ $((i % 10)) -eq 0 ]; then     curl -s -
o /dev/null -X POST http://localhost:8080/api/transferencias       -H 'Conten
t-Type: application/json'       -d '{"claveOrigen":"0021800000000000001","clav
eDestino":"0021800000000000002","monto":1}';   fi; done

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli INFO stats | grep keyspace
keyspace_hits:97
keyspace_misses:23
```

En el escenario A, sin escrituras de por medio, las 100 consultas dieron un hit ratio de 95 %, 97 % y 97 % en mis tres corridas (hits 95, 97 y 97; misses 5, 3 y 3). Los misses son el primero, que llena la caché, y los de las veces que el TTL de 30 s hizo caducar la copia mientras corría el ciclo; todo lo demás fueron hits.

En B, con una transferencia cada 10 consultas, el hit ratio bajó a 81.7 %, 82.5 % y 80.8 % (hits 98, 99 y 97; misses 22, 21 y 23). Los misses subieron de 3–5 a más de 20 porque cada transferencia borra la copia de dos cuentas, Ana y Beto, y la siguiente lectura de cada una es un miss: lee Postgres y vuelve a llenar la caché. Con 10 transferencias por corrida, salen alrededor de dos misses extra por transferencia. Entre más seguido cambian los saldos, más misses y menor hit ratio. También cuentan mis propios redis-cli GET.

3.2 Latencia y lecturas a Postgres, con y sin caché

Sin caché: `CACHE_TIPO=none docker compose up -d cuenta` (espera *healthy*). Con caché: `docker compose up -d cuenta`.

Mi predicción (¿cuánto más rápida con caché?): Hasta dies veces más rápida.

Caso	Corrida	Promedio (ms)	Mediana (ms)	p95 (ms)	Lecturas a Postgres
Con caché	1	28.16	19.02	50.83	0
Con caché	2	31.85	10.16	110.27	3
Con caché	3	18.02	11.18	47.07	4
Sin caché	1	20.65	14.46	44.10	100
Sin caché	2	20.15	12.23	57.92	200
Sin caché	3	20.23	13.06	47.15	300

Captura 3.2 — salidas de promedio / mediana / p95 (con y sin caché)

Sin caché:

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ for i in $(seq 1 100); do curl -s -o /dev/null -w '%{time_total}\n' http://
localhost:8080/api/cuentas/0021800000000000001; done | sort -n | awk '{t[NR]=$
1*1000; s+=$1*1000} END {printf "promedio=%.2f ms mediana=%.2f ms p95=%.2f ms
\n", s/NR, t[int(NR*0.5)], t[int(NR*0.95)]}'
promedio=20.65 ms mediana=14.46 ms p95=44.10 ms

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose logs cuenta | grep "leida de Postgres" | wc -l
100

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ for i in $(seq 1 100); do curl -s -o /dev/null -w '%{time_total}\n' http://
localhost:8080/api/cuentas/0021800000000000001; done | sort -n | awk '{t[NR]=$
1*1000; s+=$1*1000} END {printf "promedio=%.2f ms mediana=%.2f ms p95=%.2f ms
\n", s/NR, t[int(NR*0.5)], t[int(NR*0.95)]}'
promedio=20.15 ms mediana=12.23 ms p95=57.92 ms

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose logs cuenta | grep "leida de Postgres" | wc -l
200

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ for i in $(seq 1 100); do curl -s -o /dev/null -w '%{time_total}\n' http://
localhost:8080/api/cuentas/0021800000000000001; done | sort -n | awk '{t[NR]=$
1*1000; s+=$1*1000} END {printf "promedio=%.2f ms mediana=%.2f ms p95=%.2f ms
\n", s/NR, t[int(NR*0.5)], t[int(NR*0.95)]}'
promedio=20.23 ms mediana=13.06 ms p95=47.15 ms

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose logs cuenta | grep "leida de Postgres" | wc -l
300
```

Captura 3.2 — salidas de promedio / mediana / p95 (con y sin caché)

Con caché:

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ for i in $(seq 1 100); do curl -s -o /dev/null -w '%{time_total}\n' http://
localhost:8080/api/cuentas/002180000000000001; done | sort -n | awk '{t[NR]=$
1*1000; s+=$1*1000} END {printf "promedio=%.2f ms mediana=%.2f ms p95=%.2f ms
\n", s/NR, t[int(NR*0.5)], t[int(NR*0.95)]}'
promedio=28.16 ms mediana=19.02 ms p95=50.38 ms

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose logs cuenta | grep "leida de Postgres" | wc -l
0

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ for i in $(seq 1 100); do curl -s -o /dev/null -w '%{time_total}\n' http://
localhost:8080/api/cuentas/002180000000000001; done | sort -n | awk '{t[NR]=$
1*1000; s+=$1*1000} END {printf "promedio=%.2f ms mediana=%.2f ms p95=%.2f ms
\n", s/NR, t[int(NR*0.5)], t[int(NR*0.95)]}'
promedio=31.85 ms mediana=10.16 ms p95=110.27 ms

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose logs cuenta | grep "leida de Postgres" | wc -l
3

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ for i in $(seq 1 100); do curl -s -o /dev/null -w '%{time_total}\n' http://
localhost:8080/api/cuentas/002180000000000001; done | sort -n | awk '{t[NR]=$
1*1000; s+=$1*1000} END {printf "promedio=%.2f ms mediana=%.2f ms p95=%.2f ms
\n", s/NR, t[int(NR*0.5)], t[int(NR*0.95)]}'
promedio=18.02 ms mediana=11.18 ms p95=47.07 ms

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose logs cuenta | grep "leida de Postgres" | wc -l
4
```

La latencia no mejoró de forma clara con la caché: con caché los promedios fueron de 17.84 a 31.85 ms y las medianas de 10.16 a 19.02 ms, y sin caché los promedios quedaron en 20.15 a 20.65 ms y las medianas en 12.23 a 14.46 ms. Con caché incluso hubo un p95 de 110.27 ms en una corrida. La diferencia es chica porque en mi laptop Postgres tiene la tabla en memoria y está a un salto de red, igual que Redis, y buena parte del tiempo de cada curl es del gateway y del arranque del propio curl.

4. Los avisos en MongoDB (Sección 5 del lab)

4.1 Código explicado

documento(...): arma el aviso como un org.bson.Document: primero los campos comunes (_id, eventoId, transferenciaId, clabe, tipo, monto, saldo, fecha) y luego un switch sobre Contacto que agrega los campos de su canal (SMS: teléfono y texto; CORREO: correo, asunto y cuerpo; PUSH: token, título y cuerpo).

Como Contacto es una interfaz sellada, si al switch le falta un canal no compila; si mañana se agrega uno nuevo, el error sale en compilación y no en producción. El monto y el saldo van en Decimal128 y no en

double porque el punto flotante no representa bien los decimales (0.1 + 0.2 no da 0.3), y con dinero eso no es aceptable. La fecha es la del evento y no la de hoy, para que el aviso refleje cuándo ocurrió la transferencia aunque se procese tarde. El _id lo armé como eventoId:clabe.

registrar(evento) — inserta uno por uno y atrapa DuplicateKeyException: registrar(evento) arma el documento de cada destinatario (origen y destino) y los inserta uno por uno. Si Mongo responde DuplicateKeyException porque el _id ya existe, ese aviso ya estaba guardado: no cuenta como error, no se suma a los nuevos y se sigue con el siguiente.

Los inserto de uno en uno y no todos juntos porque un insert múltiple se detiene en el primer _id repetido; si el proceso se cayó después de guardar el primer aviso, la reentrega todavía tiene que poder guardar el segundo.

crearIndice() y ultimos(clabe) — Query, Sort y limit: crearIndice() crea al arrancar el índice clabe_fecha con clabe ascendente y fecha descendente, que sigue la forma de la consulta de ultimos. ultimos(clabe) arma una Query con Criteria.where("clabe").is(clabe) (el WHERE), la ordena con Sort por fecha descendente (el ORDER BY) y la limita a 10 con limit(ULTIMOS) (el LIMIT). Sin el índice, Mongo tendría que recorrer toda la colección y ordenar en memoria.

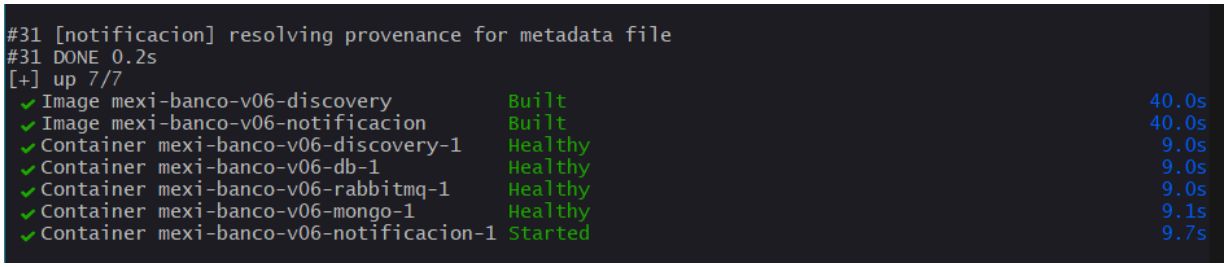
ConsumidorDeTransferencias: En ConsumidorDeTransferencias.recibir cambié una sola línea: en lugar de guardar los avisos en la bandeja de Postgres con notificaciones.avisarDeTransferencia(evento).size(), ahora llamo a bitacora.registrar(evento). Guardo solo en Mongo y no en los dos porque sería una doble escritura sin transacción común: si una falla, las dos bases quedan distintas. Es el mismo problema que resolvió el Outbox en la S09; los avisos se mudan, no s

En **ConsumidorDeTransferencias.recibir** cambié una sola línea: en lugar de guardar los avisos en la bandeja de Postgres con notificaciones.avisarDeTransferencia(evento).size(), ahora llamo a bitacora.registrar(evento). Guardo solo en Mongo y no en los dos porque sería una doble escritura sin transacción común: si una falla, las dos bases quedan distintas.

4.2 Índice creado al arrancar

Comando: `docker compose up --build -d notificacion` y `docker compose logs notificacion | grep avisos`.

📷 Captura 4.2 — [avisos] indice clabe_fecha listo en avisos



📷 Captura 4.2 — [avisos] indice clabe_fecha listo en avisos

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose logs notificacion | grep avisos
notificacion-1  | 2026-11-06T23:40:55.776Z  INFO 1 --- [notificacion] [           main] m.m.n.avisos.Bi
tacoraDeAvisos           : [avisos] indice clabe_fecha listo en avisos
```

4.3 Las tres formas de aviso

Transferencias de Ana a Beto y de Ana a Carla; luego `db.avisos.find().sort({fecha: 1})` y `db.avisos.find({telefono: {$exists: true}}, {clabe: 1, canal: 1, telefono: 1})`.

Captura 4.3a — los cuatro avisos: 2 SMS (Ana), 1 CORREO (Beto), 1 PUSH (Carla)

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find().sort({fecha: 1})'
[
  {
    _id: '512a3df4-a2df-47f1-8aa2-3975fe8ea47c:002180000000000001',
    eventoId: '512a3df4-a2df-47f1-8aa2-3975fe8ea47c',
    transferenciaId: Long('2'),
    clabe: '002180000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('10'),
    saldo: Decimal128('980.00'),
    fecha: ISODate('2026-11-07T04:31:27.045Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 10, saldo resultante 980.00'
  },
  {
    _id: '512a3df4-a2df-47f1-8aa2-3975fe8ea47c:002180000000000002',
    eventoId: '512a3df4-a2df-47f1-8aa2-3975fe8ea47c',
    transferenciaId: Long('2'),
    clabe: '002180000000000002',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('10'),
    saldo: Decimal128('1020.00'),
    fecha: ISODate('2026-11-07T04:31:27.045Z'),
    canal: 'CORREO',
    correo: 'beto@correo.mx',
    asunto: 'Aviso de movimiento',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 10, saldo resultante 1020.00'
  },
  {
    _id: '8a9d2621-8dbb-4760-ae8b-1b67d64b9c29:002180000000000001',
    eventoId: '8a9d2621-8dbb-4760-ae8b-1b67d64b9c29',
    transferenciaId: Long('3'),
    clabe: '002180000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('10'),
    saldo: Decimal128('970.00'),
    fecha: ISODate('2026-11-07T04:31:42.849Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 10, saldo resultante 970.00'
  },
  {
    _id: '8a9d2621-8dbb-4760-ae8b-1b67d64b9c29:002180000000000003',
    eventoId: '8a9d2621-8dbb-4760-ae8b-1b67d64b9c29',
    transferenciaId: Long('3'),
    clabe: '002180000000000003',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('10'),
    saldo: Decimal128('1010.00'),
    fecha: ISODate('2026-11-07T04:31:42.849Z'),
    canal: 'PUSH',
    token: 'fcm:demo-002180000000000003',
    titulo: 'Notificación MexiBanco',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 10, saldo resultante 1010.00'
  }
```

Captura 4.3b — solo los avisos con telefono (los dos de Ana)

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({telefono: {$exists: true}},
 {clabe: 1, canal: 1, telefono: 1})'
[
  {
    _id: '512a3df4-a2df-47f1-8aa2-3975fe8ea47c:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: '8a9d2621-8dbb-4760-ae8b-1b67d64b9c29:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  }
]
```

4.4 El endpoint devuelve los últimos 10

Doce transferencias de 1 peso de Ana a Beto y `GET /api/notificaciones/002180000000000002`.

📷 Captura 4.4 — respuesta con exactamente 10 avisos CORREO, del más nuevo al más viejo

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ for i in $(seq 1 12); do curl -s -o /dev/null -X POST http://localhost:8080/api/transferencias -H 'Co
ntent-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":1}'; done

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ curl -s http://localhost:8080/api/notificaciones/002180000000000002
[{"_id":"9cc37118-c01f-4ca7-9cb8-b83f0ded468e:002180000000000002","eventoId":"9cc37118-c01f-4ca7-9cb8-b
83f0ded468e","transferenciaId":15,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":
1,"saldo":1032.00,"fecha":"2026-11-07T04:37:48.919+00:00","canal":"CORREO","correo":"beto@correo.mx","a
sunto":"Aviso de movimiento","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 1032.00"},{"_id":
"45f044e3-4949-40a7-90cf-1131b2da61ab:002180000000000002","eventoId":"45f044e3-4949-40a7-90cf-1131b2da6
1ab","transferenciaId":14,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo
":1031.00,"fecha":"2026-11-07T04:37:48.564+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"
Aviso de movimiento","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 1031.00"},{"_id":"f04f0d0
0-8e9c-4a55-9379-4a9c2c8d7506:002180000000000002","eventoId":"f04f0d00-8e9c-4a55-9379-4a9c2c8d7506","tr
ansferenciaId":13,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":1030.0
0,"fecha":"2026-11-07T04:37:48.273+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Aviso de
 movimiento","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 1030.00"},{"_id":"9bbeaba5-2cec-4
911-88db-83ecf9b48b74:002180000000000002","eventoId":"9bbeaba5-2cec-4911-88db-83ecf9b48b74","transferen
ciaId":12,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":1029.00,"fecha
":"2026-11-07T04:37:47.977+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Aviso de movimie
nto","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 1029.00"},{"_id":"6c8d4abf-dfe1-4c8a-a30e
-dc12e3966428:002180000000000002","eventoId":"6c8d4abf-dfe1-4c8a-a30e-dc12e3966428","transferenciaId":1
1,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":1028.00,"fecha":"2026-
11-07T04:37:47.686+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Aviso de movimiento","cu
erpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 1028.00"},{"_id":"e2e078e1-d6b4-43a8-ad74-bd05cbe
aa596:002180000000000002","eventoId":"e2e078e1-d6b4-43a8-ad74-bd05cbeaa596","transferenciaId":10,"clabe
":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":1027.00,"fecha":"2026-11-07T04
:37:47.328+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Aviso de movimiento","cuerpo":"T
RANSFERENCIA_RECIBIDA por 1, saldo resultante 1027.00"},{"_id":"1bac9edb-d50d-4122-b676-3fb9101c4dcc:00
2180000000000002","eventoId":"1bac9edb-d50d-4122-b676-3fb9101c4dcc","transferenciaId":9,"clabe":"002180
000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":1026.00,"fecha":"2026-11-07T04:37:47.08
8+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Aviso de movimiento","cuerpo":"TRANSFEREN
CIA_RECIBIDA por 1, saldo resultante 1026.00"},{"_id":"a921eede-313c-4d73-be4d-23e90a386346:00218000000
0000002","eventoId":"a921eede-313c-4d73-be4d-23e90a386346","transferenciaId":8,"clabe":"002180000000000
002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":1025.00,"fecha":"2026-11-07T04:37:46.723+00:00",
"canal":"CORREO","correo":"beto@correo.mx","asunto":"Aviso de movimiento","cuerpo":"TRANSFERENCIA_RECIB
IDA por 1, saldo resultante 1025.00"},{"_id":"7f6be11c-dcb2-4237-a365-2d405d708a79:002180000000000002",
"eventoId":"7f6be11c-dcb2-4237-a365-2d405d708a79","transferenciaId":7,"clabe":"002180000000000002","tip
o":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":1024.00,"fecha":"2026-11-07T04:37:46.238+00:00","canal":"
CORREO","correo":"beto@correo.mx","asunto":"Aviso de movimiento","cuerpo":"TRANSFERENCIA_RECIBIDA por 1
, saldo resultante 1024.00"},{"_id":"5ea109b6-f628-40f1-86eb-dce96888554c:002180000000000002","eventoId
":"5ea109b6-f628-40f1-86eb-dce96888554c","transferenciaId":6,"clabe":"002180000000000002","tipo":"TRANS
FERENCIA_RECIBIDA","monto":1,"saldo":1023.00,"fecha":"2026-11-07T04:37:45.961+00:00","canal":"CORREO","
correo":"beto@correo.mx","asunto":"Aviso de movimiento","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo r
esultante 1023.00"}]
```

4.5 El índice trabaja (explain)

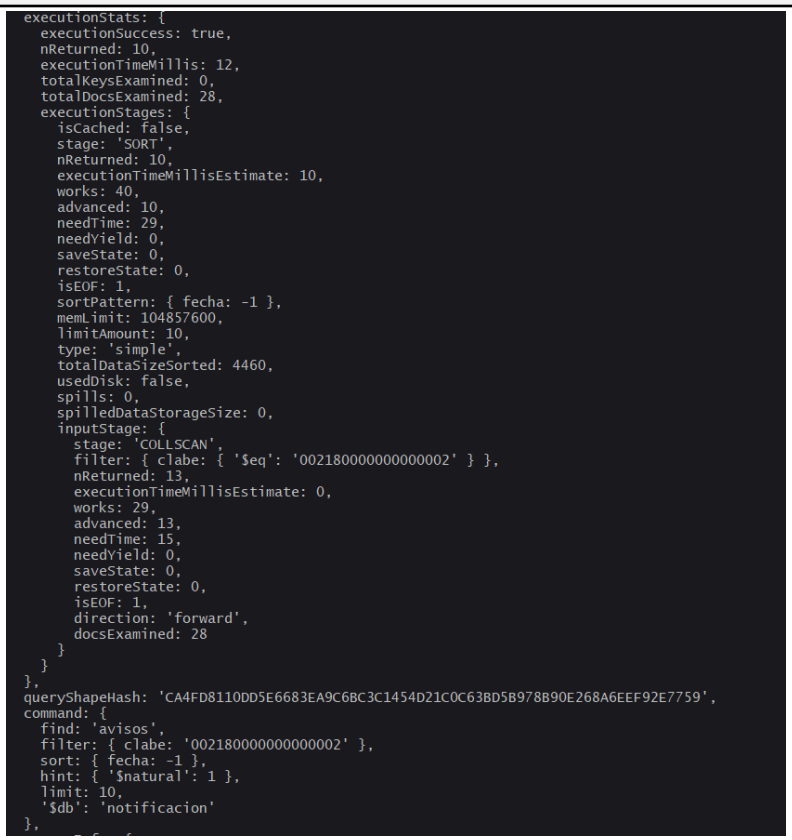
Captura 4.5a — db.avisos.getIndexes()

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.getIndexes()'
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { clabe: 1, fecha: -1 }, name: 'clabe_fecha' }
]
```

Captura 4.5b — explain("executionStats") CON índice (IXSCAN, totalDocsExamined)

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({clabe: "002180000000000002"
}).sort({fecha: -1}).limit(10).hint({$natural: 1}).explain("executionStats")'
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'notificacion.avisos',
    parsedQuery: { clabe: { '$eq': '002180000000000002' } },
    indexFilterSet: false,
    queryHash: 'C73271DB',
    planCacheShapeHash: 'C73271DB',
    planCacheKey: '501A071D',
    optimizationTimeMillis: 3,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'SORT',
      sortPattern: { fecha: -1 },
      memLimit: 104857600,
      limitAmount: 10,
      type: 'simple',
      inputStage: {
        stage: 'COLLSCAN',
        filter: { clabe: { '$eq': '002180000000000002' } },
        direction: 'forward'
      }
    },
    rejectedPlans: []
  },
```

Captura 4.5b — explain("executionStats") CON índice (IXSCAN, totalDocsExamined)



```
executionStats: {
  executionSuccess: true,
  nReturned: 10,
  executionTimeMillis: 12,
  totalKeysExamined: 0,
  totalDocsExamined: 28,
  executionStages: {
    isCached: false,
    stage: 'SORT',
    nReturned: 10,
    executionTimeMillisEstimate: 10,
    works: 40,
    advanced: 10,
    needTime: 29,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    sortPattern: { fecha: -1 },
    memLimit: 104857600,
    limitAmount: 10,
    type: 'simple',
    totalDataSizeSorted: 4460,
    usedDisk: false,
    spills: 0,
    spilledDataStorageSize: 0,
    inputStage: {
      stage: 'COLLSCAN',
      filter: { clabe: { '$eq': '002180000000000002' } },
      nReturned: 13,
      executionTimeMillisEstimate: 0,
      works: 29,
      advanced: 13,
      needTime: 15,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      direction: 'forward',
      docsExamined: 28
    }
  }
},
queryShapeHash: 'CA4FD8110DD5E6683EA9C6BC3C1454D21C0C63BD5B978B90E268A6EEF92E7759',
command: {
  find: 'avisos',
  filter: { clabe: '002180000000000002' },
  sort: { fecha: -1 },
  hint: { '$natural': 1 },
  limit: 10,
  '$db': 'notificacion'
},
```



```
queryShapeHash: 'CA4FD8110DD5E6683EA9C6BC3C1454D21C0C63BD5B978B90E268A6EEF92E7759',
command: {
  find: 'avisos',
  filter: { clabe: '002180000000000002' },
  sort: { fecha: -1 },
  hint: { '$natural': 1 },
  limit: 10,
  '$db': 'notificacion'
},
serverInfo: {
  host: 'e5ecbccc0faf',
  port: 27017,
  version: '8.0.34',
  gitVersion: '51db4515a6597aedcf6188ee4e48a27e65b065ae'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFrameworkControl: 'trySbeRestricted',
  internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
},
ok: 1
}
```

Captura 4.5c — explain SIN índice con hint({$natural: 1}) (COLLSCAN, totalDocsExamined)

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({clabe: "002180000000000002"
}).sort({fecha: -1}).limit(10).hint({$natural: 1}).explain("executionStats")'
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'notificacion.avisos',
    parsedQuery: { clabe: { '$eq': '002180000000000002' } },
    indexFilterSet: false,
    queryHash: 'C73271DB',
    planCacheShapeHash: 'C73271DB',
    planCacheKey: '501A071D',
    optimizationTimeMillis: 3,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'SORT',
      sortPattern: { fecha: -1 },
      memLimit: 104857600,
      limitAmount: 10,
      type: 'simple',
      inputStage: {
        stage: 'COLLSCAN',
        filter: { clabe: { '$eq': '002180000000000002' } },
        direction: 'forward'
      }
    },
    rejectedPlans: []
  },
```

```
  executionStats: {
    executionSuccess: true,
    nReturned: 10,
    executionTimeMillis: 12,
    totalKeysExamined: 0,
    totalDocsExamined: 28,
    executionStages: {
      isCached: false,
      stage: 'SORT',
      nReturned: 10,
      executionTimeMillisEstimate: 10,
      works: 40,
      advanced: 10,
      needTime: 29,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      sortPattern: { fecha: -1 },
      memLimit: 104857600,
      limitAmount: 10,
      type: 'simple',
      totalDataSizeSorted: 4460,
      usedDisk: false,
      spills: 0,
      spilledDataStorageSize: 0,
      inputStage: {
        stage: 'COLLSCAN',
        filter: { clabe: { '$eq': '002180000000000002' } },
        nReturned: 13,
        executionTimeMillisEstimate: 0,
        works: 29,
        advanced: 13,
        needTime: 15,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 1,
        direction: 'forward',
        docsExamined: 28
      }
    }
  },
```

Captura 4.5c — explain SIN índice con hint({$natural: 1}) (COLLSCAN, totalDocsExamined)


```
  queryShapeHash: 'CA4FD81110DD5E6683EA9C6BC3C1454D21C0C63BD5B978B90E268A6EEF92E7759',
  command: {
    find: 'avisos',
    filter: { clabe: '002180000000000002' },
    sort: { fecha: -1 },
    hint: { '$natural': 1 },
    limit: 10,
    '$db': 'notificacion'
  },
  serverInfo: {
    host: 'e5ecbccc0faf',
    port: 27017,
    version: '8.0.34',
    gitVersion: '51db4515a6597aedcf6188ee4e48a27e65b065ae'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}
```


5. Rompe algo: el evento duplicado (Sección 6 del lab)

Publicar **dos veces** el mismo evento (`eventoId` `11111111-2222-4333-8444-555555555555`, CLABEs `...091` y `...092`) por la API de RabbitMQ.

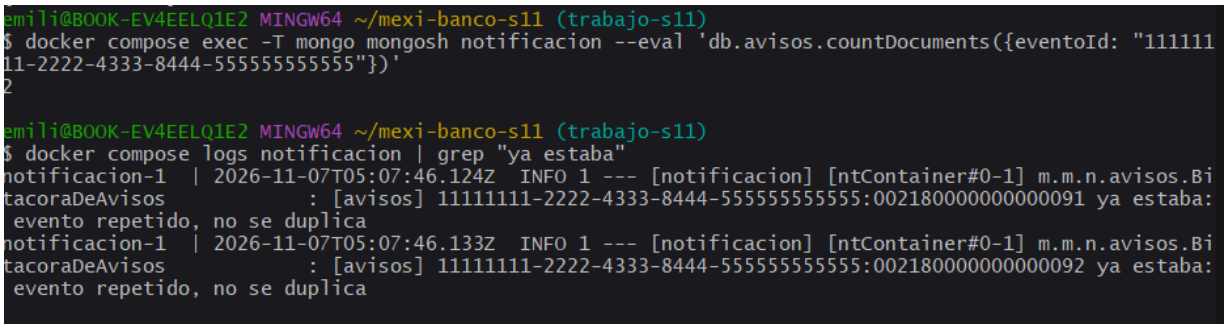
Mi predicción (dos eventos iguales, dos avisos cada uno: ¿cuántos documentos quedan?): quedan dos documentos.

Captura 5.1 — las dos respuestas {"routed":true}


```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco-s11 (trabajo-s11)
$ curl -u mexibanco:mexibanco -X POST http://localhost:15672/api/exchanges/%2F/transferencias/publish \

  -H 'Content-Type: application/json' \
  -d '{"properties":{"delivery_mode":2,"content_type":"application/json"},"routing_key":"transferencia.
realizada","payload_encoding":"string","payload":"{\"eventoId\":\"11111111-2222-4333-8444-555555555555\
",\"tipo\":\"TransferenciaRealizada\",\"transferenciaId\":9001,\"clabeOrigen\":\"002180000000000091\",\
"clabeDestino\":\"002180000000000092\",\"monto\":25,\"saldoOrigen\":475,\"saldoDestino\":525,\"fecha\":
\"2026-10-31T13:00:00Z\"}"}'
{"routed":true}
```

Captura 5.2 — countDocuments del evento y las líneas "ya estaba" del log



Explicación: ¿por qué no se duplican los avisos? (rol del `_id`, `DuplicateKeyException` y entrega *al menos una vez*):

Usar la combinación eventoId:clabe como _id en Mongo funciona como una llave de idempotencia perfecta. Si RabbitMQ nos entrega el mismo evento dos veces (por su política de "al menos una vez" que vimos en la S09), Mongo rechaza el duplicado y lanza una DuplicateKeyException. Nuestro método registrar simplemente atrapa el error y lo ignora silenciosamente. Así garantizamos que no se dupliquen los avisos (quedan solo los 2 documentos reales y no 4), resolviendo el problema de las repeticiones sin tener que armar una tabla extra para llevar el control.

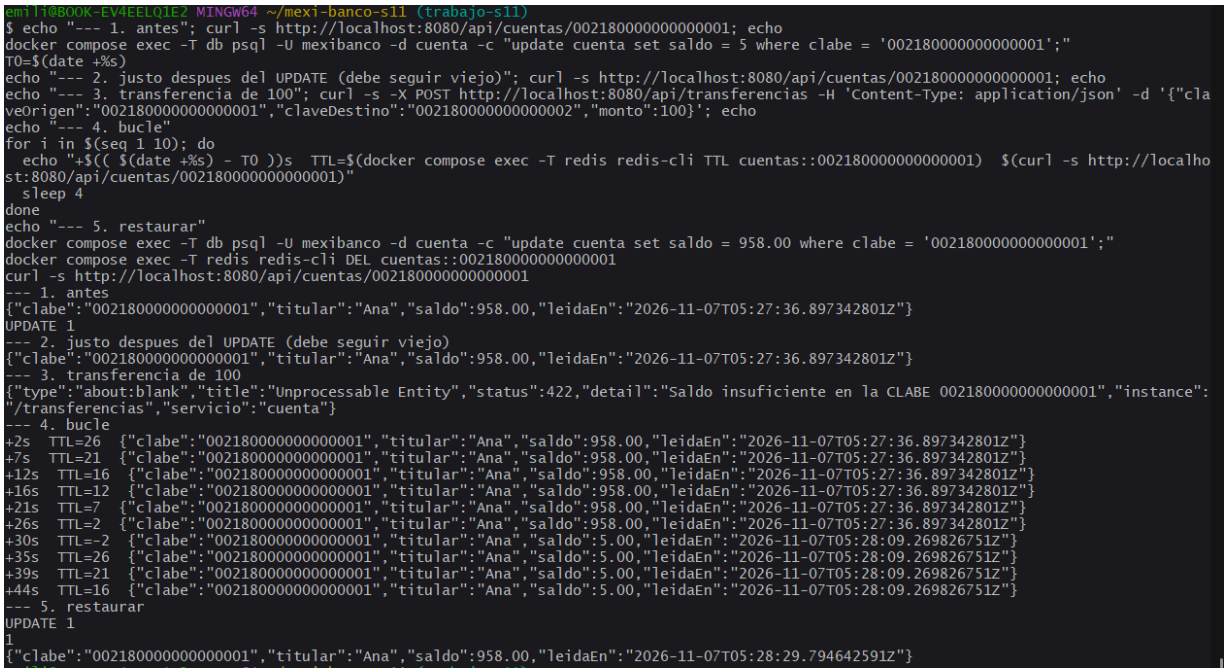
6. Rompe algo: el dato viejo

Cambiar el saldo de Ana directo en Postgres (`update cuenta set saldo = 5 ...`) para que la caché quede desactualizada.

Dato	Valor
Saldo de Ana antes (en la caché)	958.00
Saldo real en Postgres tras el `UPDATE`	5
Segundos que el cliente vio el saldo viejo	30 s (vio 958.00 hasta +26 s y 5.00 desde + 30s)
Respuesta de la transferencia de 100	422 Unprocessable Entity: "Saldo insuficiente en la CLABE 002180000000000001"

Mi predicción (¿pasa la transferencia de 100 mientras la app muestra el saldo viejo?): No, no va a pasar.

Captura 6.1 — consulta inicial y UPDATE directo en Postgres, ciclo de 8 iteraciones: TTL bajando, saldo viejo y luego saldo 5 con leidaEn nuevo y transferencia de 100 ((respuesta 422 "Saldo insuficiente")



7. Preguntas de cierre

1. ¿Por qué MongoDB para los avisos y no para el saldo?

MongoDB sirve para los avisos porque cada aviso tiene la forma de su canal (SMS, correo o push), se escribe una vez, nunca se actualiza y se lee completo, así que un documento por aviso es natural y evita columnas vacías o una tabla por canal. El saldo, en cambio, es dinero: necesita transacciones, consistencia fuerte y control de concurrencia (@Version) para que dos cargos simultáneos no lo corrompan, y eso lo da PostgreSQL.

2. ¿Qué hit ratio mediste en A y en B, y por qué cambia? Con tus números de latencia y de lecturas a Postgres: ¿qué compraste con la caché, velocidad o capacidad?

En A mi hit ratio fue de 95 %, 97 % y 97 % (tres corridas) y en B de 81.7 %, 82.5 % y 80.8 %. En B baja porque cada transferencia borra la copia de Ana y de Beto, y la siguiente lectura de cada uno es un miss. Sobre lo que compré: la latencia casi no cambió (medianas de 10.16 a 19.02 ms con caché y de 12.23 a 14.46 ms sin ella), pero las lecturas a Postgres pasaron de 100 por corrida sin caché a entre 0 y 3 por corrida con caché. Compré capacidad y no velocidad: Postgres queda libre para los cargos y abonos.

3. ¿Qué pasaría si la autorización del cargo leyera el saldo de la caché?

Si la autorización del cargo leyera el saldo de la caché, podría aprobar o rechazar un movimiento con un saldo de hace hasta 30 s. En la Parte 4 el saldo real era 5 pero la caché seguía mostrando el anterior; con la caché en la decisión, la transferencia de 100 se habría aprobado y Ana habría quedado con saldo negativo. Por eso el cargo lee siempre de Postgres.

4. Al aplicar un movimiento, ¿por qué se hace `DEL` de la copia y no `SET` con el saldo nuevo? ¿Y por qué después del commit y no antes?

Se hace DEL y no SET porque si dos movimientos casi simultáneos hicieran SET con su saldo nuevo, podrían llegar a Redis en distinto orden y dejar un saldo que no es el último. Con DEL no hay que adivinar nada: la siguiente consulta lee Postgres, que es la fuente de verdad.

Y es después del commit porque si se borrara antes, otra consulta podría leer en esa ventana el saldo viejo de Postgres (el cambio aún no está confirmado) y volver a guardarlo en la caché, dejando la copia vieja viva otros 30 s. Para eso uso transactionAware().